

- [51] T. Mitchell, W. Cohen, E. Hruschka, P. Talukdar, B. Yang, J. Betteridge, A. Carlson, B. Dalvi, M. Gardner, B. Kisiel, *et al.*, “Never-ending learning,” *Communications of the ACM*, vol. 61, no. 5, pp. 103–115, 2018.
- [52] D. Ha and J. Schmidhuber, “World models,” *arXiv preprint arXiv:1803.10122*, 2018.
- [53] C. M. Bishop, “Mixture density networks,” 1994.

Appendix

A More Details on Problem Generation with LOGO

As described in Section 2.1, three types of BONGARD-LOGO problems consist of two types of shapes. Free-form shape problems only contain *free-form shapes*, while both basic shape problems and abstract shape problems are composed of samples from 627 *human-designed shapes*. Since we have discussed how to use rendered shapes (i.e., *free-form* and *human-designed*) to form different types of problems, we will next talk about the process of generating each type of shapes.

Free-form shapes A free-form shape is generated in the following steps: 1) Decide what length of action programs a shape has. 2) Randomly and independently sample each base action and its corresponding three moving arguments (i.e., *moving type*, *moving length* and *moving angle*) in sequence, resulting in a tentative action program. 3) Execute the above tentative action program in the shape renderer (i.e., turtle graphics) to generate the shape. Sometimes, the strokes of the rendered shape are heavily overlapped with each other, making the shape difficult to recognize. If it has happened, we go back to step 2) and repeat the process until either there is no heavy overlapping or the maximum number of steps has reached. Note that for the free-form shape generation, the positive and negative images would be easily indistinguishable if the moving length and angle of each action have continuous values. Therefore, we further constrain the moving length and angle to be only sampled from a set of well-separated discrete values.

Human-designed shapes Among 627 human-designed shapes, each one is paired with a ground-truth action strokes. All these shapes are stored in a dictionary with a key-value pair: (*shape name*, *action strokes*). The only difference between *action program* and *action strokes* is that the latter does not specify the *moving type*, as both basic shape and abstract shape problems treat it as one of the nuisances. Therefore, a human-designed shape is generated in the following steps: 1) Randomly sample a shape name from a predetermined subset of 627 shape names. Note that for basic shape problems, the predetermined subset is depicted by shape categories (such as triangles, squares, etc.), while for abstract shape problems, the predetermined subset is depicted by abstract attributes (such as convex, symmetric, etc.). 2) Infer its corresponding *action strokes* by looking up the dictionary and add randomly sampled *moving type* to each action, resulting in the final action program. 3) Execute the above action program in the shape renderer (i.e., turtle graphics) to generate the shape.

During the generation process of both two types of shapes, we will randomize the initial starting point and initial moving angle of each shape, and the size of unit length, such that shape position, shape orientation and shape size are all considered as nuisances. It demands a perception that is both rotation-invariant and scale-invariant.

B More Details on Methods

In this section, we provide more details on state-of-the-art (SOTA) meta-learning approaches and other strong baselines. As BONGARD-LOGO problems can be cast as a *two-way six-shot* few-shot classification problem, where meta-learning has been a standard framework [30, 4], we first discuss different SOTA meta-learning methods:

SNAIL [19] SNAIL is a memory-based meta-learning method. It proposed a class of simple and generic meta-learner architectures that use a novel combination of temporal convolutions and soft attention, with the former to aggregate information from past experience and the latter to pinpoint specific pieces of information.

ProtoNet [20] ProtoNet is a metric-based meta-learning method. It proposed a simple method called prototypical networks based on the idea that we can represent each class by the mean of its examples in a representation space learned by a neural network. In the learned metric space, classification can be performed by computing distances to prototype representations of each class.

MetaOptNet [21] MetaOptNet is an optimization-based meta-learning method. It proposed to learn the feature representation that can generalize well for a linear support vector machine (SVM) classifier.

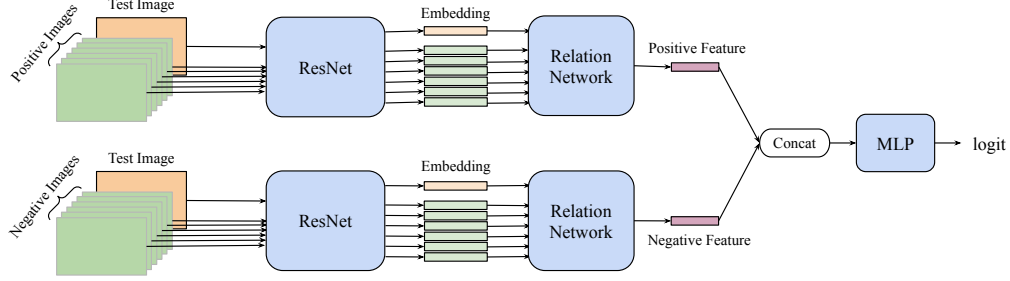


Figure 3: WReN-Bongard, where the key idea is to use relation network to form representations of pair-wise relation between each context (i.e., positive or negative) image and a given test image, and between context images themselves. Note that the ‘logit’ will pass into a sigmoid function for binary classification.

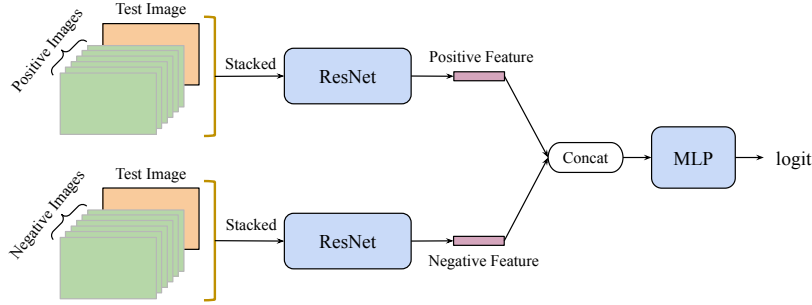


Figure 4: CNN-Baseline, where we first stack the given test image and all six positive (or all six negative images) to form a “stacked image” with input seven channels. The two stacked images pass into ResNet to extract the respective features for obtaining the final logit. Similarly, the ‘logit’ will pass into a sigmoid function for binary classification.

By exploiting the dual formulation and KKT conditions, MetaOptNet enabled a computational and memory efficient meta-learning with higher embedding dimensions for improved performance.

ANIL [22] ANIL is an optimization-based meta-learning method. It is a simplification of MAML [5] where we remove the inner loop for all but the (task-specific) head of the underlying neural network. ANIL matched MAML’s performance on several few-shot image classification benchmarks with significantly improved computational and memory efficiency.

Meta-Baseline-SC & -MoCo Meta-Baseline [23] is a new competitive baseline designed to investigate the role of feature representations in few-shot learning. It proposed to pre-train a classifier on all base classes and then meta-learn with a nearest-centroid based few-shot classification algorithm. Because there is no base class in BONGARD-LOGO, a supervised image classification pre-training is infeasible. Instead, we introduced two variants: 1) Meta-Baseline-SC, where we meta-train the Meta-Baseline from scratch, and 2) Meta-Baseline-MoCo, where we first use an unsupervised contrastive learning method MoCo [31] to pre-train the backbone model and then apply meta-training.

Besides, we consider two strong non-meta-learning baselines for comparison.

WReN-Bongard WReN [17] is a new architecture, designed to encourage reasoning, by solving visual IQ tests, the Raven-style Progressive Matrices (RPMs) [32]. It has achieved strong performances on the RPMs. The idea of WReN is to form representations of pair-wise relation between each context and a given choice candidate, and between contexts themselves. We apply its relation network module to our problems. To do so, we develop a variant of WReN, named WReN-Bongard, by adapting WReN to Bongard problems. Figure 3 shows the architecture of WReN-Bongard.

CNN-Baseline CNN is a standard deep learning baseline of image classification without meta-learning or relation reasoning. The idea of CNN-Baseline is to stack the test image in each problem alongside all six positive images and all six negative images, respectively, to form two “stacked images” with seven input channels. This way, we convert the few-shot learning problem to a conventional binary image classification problem. Figure 4 shows the architecture of CNN-Baseline.